One Primitive: Running Conway's Game of Life and a Transformer on Apple's Neural Engine via Convolution

Alvaro Videla
Microsoft

April 2026

Abstract

We show that a single operation—convolution—suffices to run both Conway's Game of Life (GOL) and a full transformer language model on Apple's Neural Engine (ANE). GOL is encoded as a 3×3 convolution with ReLU-based rule evaluation, achieving 2.3 billion cell-generations per second on a 1024×1024 grid (2.5× faster than CPU). The Qwen2.5-0.5B transformer is mapped entirely to `Conv2d(1×1)` operations, with every linear projection expressed as a 1×1 convolution, achieving 171 tokens/sec decode throughput—within 2× of llama.cpp on Metal GPU, using a compute unit (ANE) that conventional LLM engines ignore. Both systems share the same silicon: the ANE's 16-core convolution engine. We describe the algebraic encoding of GOL rules into convolution + ReLU, the transformer-to-convolution mapping, and a series of optimizations—book-driven host elimination, pre-allocated buffers, fixed-shape compiled models, and CoreML stateful KV caching—that improved decode throughput by 19× from initial implementation to final system.

1 Introduction

Apple's Neural Engine (ANE), present in every Apple Silicon chip since A11, is a hardware accelerator optimized for convolution, element-wise arithmetic, and activation functions. Unlike the GPU—which excels at arbitrary parallel kernels—the ANE achieves peak throughput on a narrow set of operations: small-kernel convolutions, ReLU, and channel-wise reductions.

This paper demonstrates that convolution alone is a sufficient primitive to implement two seemingly different computations on the ANE:

1. **Conway's Game of Life** — a cellular automaton where each generation is a 3×3 convolution counting neighbors, followed by ReLU-based rule evaluation.

2. **Transformer inference** — a Qwen2.5-0.5B language model where every matrix multiplication is expressed as a `Conv2d(1×1)` operation.

The connection is not metaphorical. Both computations compile to the same ANE instruction stream: convolution kernels dispatched to the same 16-core engine. The key insight is that the ANE's convolution unit can execute any linear operation when the spatial dimensions are 1×1 (reducing to matrix–vector multiply) and any local cellular rule when the spatial dimensions match the grid.

Section 2 presents the GOL encoding. Section 3 describes the transformer mapping. Section 4 details the optimization journey. Section 5 presents benchmarks. Section 6 discusses related work.

2 Game of Life as Convolution

2.1 The Encoding

Conway's Game of Life operates on a binary grid where each cell is alive (1) or dead (0). The rules depend on the count of alive neighbors (out of 8):

- **Birth**: A dead cell with exactly 3 neighbors becomes alive.
- **Survival**: An alive cell with 2 or 3 neighbors stays alive.
- **Death**: All other cells die (or remain dead).

We encode one GOL generation as a convolution followed by ReLU-based rule evaluation.

Step 1: Convolution. Apply a 3×3 kernel:

$$K = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 9 & 1 \\ 1 & 1 & 1 \end{bmatrix} \tag{1}$$

The center weight 9 (rather than 0 or 1) encodes the cell's own state into the output. For a cell at position (r, c) with value $x_{r,c} \in \{0, 1\}$ and neighbor count n:

$$s = n + 9 \cdot x_{r,c} = \begin{cases} n & \text{if dead} \quad (s \in [0, 8]) \\ n + 9 & \text{if alive} \quad (s \in [9, 17]) \end{cases} \tag{2}$$

This separates the dead and alive cases into non-overlapping integer ranges, enabling branchless rule evaluation.

Step 2: ReLU rule evaluation. We implement an exact equality test using ReLU:

$$\mathrm{eq}(s, k) = \mathrm{relu}\big(1 - (\mathrm{relu}(s - k) + \mathrm{relu}(k - s))\big) \tag{3}$$

This yields exactly 1 when $s = k$ and 0 otherwise (for non-negative integers). The GOL rules become:

$$x'_{r,c} = \mathrm{clamp}\big(\mathrm{eq}(s, 3) + \mathrm{eq}(s, 11) + \mathrm{eq}(s, 12),\ 0,\ 1\big) \tag{4}$$

where $s = 3$ captures birth (dead, 3 neighbors), $s = 11$ captures survival with 2 neighbors (alive, $2 + 9$), and $s = 12$ captures survival with 3 neighbors (alive, $3 + 9$).

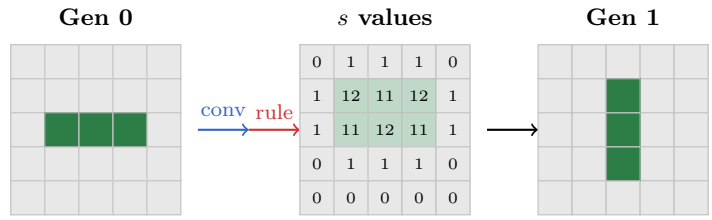

Figure 1: Blinker oscillator through one GOL generation. Left: the horizontal blinker (3 alive cells). Center: s values after the 3×3 convolution with kernel K. Values ≥ 9 (shaded) indicate alive cells. Only positions with $s \in \{3, 11, 12\}$ survive the ReLU rule. Right: the vertical blinker.

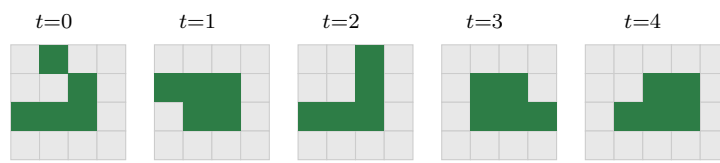

Figure 2: The glider spaceship translating diagonally over 4 generations, each computed by one conv+ReLU layer on ANE. After 4 generations the pattern has moved one cell down and one cell right.

Multi-generation pipelining. Stacking N copies of the conv+rule layer produces a single network that computes N generations in one forward pass, pipelined through the ANE's 16 cores.

2.2 Visual Example

Figure 1 shows the blinker oscillator (period 2) through one generation of the convolution encoding.

3 Transformer as Convolution

The Game of Life section showed that a 3×3 convolution can implement a cellular automaton. This section shows that the *same* convolution hardware can run a neural network—because a neural network is, at bottom, a sequence of matrix–vector products interleaved

with simple nonlinearities, and a 1×1 convolution *is* a matrix–vector product.

3.1 What a Transformer Actually Computes

A transformer processes one token at a time through a stack of identical layers. Stripping away the notation, each layer performs five steps:

1. **Normalize** the input vector (RMSNorm: divide each element by the root-mean-square of all elements).
2. **Project** the vector into queries, keys, and values via three matrix multiplications: $q = W_Q x$, $k = W_K x$, $v = W_V x$.
3. **Attend**: compute a weighted combination of past values using the dot product of queries and keys as weights (attention).
4. **Project** the result back: $o = W_O \cdot \text{attn}$.
5. **Feed-forward network**: two more matrix multiplications with a nonlinearity (SiLU) in between: $\text{FFN}(x) = W_{\text{down}}(\text{SiLU}(W_{\text{gate}}\, x) \odot W_{\text{up}}\, x)$.

After 24 such layers, one final matrix multiplication (W_{head}) maps the 896-dimensional vector to a probability distribution over 151,936 vocabulary tokens.

The key observation: **steps 1, 2, 4, and 5 are all matrix–vector products** $y = Wx$. The nonlinearities (SiLU, softmax, RMSNorm) are element-wise and cheap. The computational cost of a transformer is dominated by these matrix multiplications—seven per layer, plus the final LM head, totaling 169 matmuls per token for Qwen2.5-0.5B.

If we can express matrix–vector multiplication as convolution, we can run the entire transformer on the ANE's convolution engine.

3.2 Why 1×1 Convolution Is Matrix Multiplication

A standard 2D convolution slides a kernel over a spatial grid. At each position, it computes a weighted sum of a local neighborhood across all input channels. But consider what happens when the spatial dimensions are 1×1—a single "pixel":

- There is *no spatial extent* to slide over. The kernel visits exactly one position.
- The "neighborhood" is just that single pixel—all that remains is the channel dimension.
- The weighted sum across C_{in} input channels, repeated for each of C_{out} output channels, is:

$$y_i = \sum_{j=1}^{C_{\text{in}}} W_{ij} \cdot x_j \quad \text{for } i = 1, \ldots, C_{\text{out}} \tag{5}$$

This is exactly the definition of matrix–vector multiplication $y = Wx$. The convolution hardware computes it natively; we need only reshape the data.

Data reshaping. A transformer's hidden state is a 1D vector of dimension $d = 896$. The ANE expects a 4D tensor in (B, C, H, W) format. The mapping is:

$$\underbrace{x}_{\text{shape (896,)}} \longrightarrow \underbrace{x'}_{\text{shape (1, 896, 1, 1)}}$$

The 896 elements become 896 "channels" of a single 1×1 "image." A `Conv2d(896, 1152, kernel_size=1)` then produces a $(1, 1152, 1, 1)$ output—exactly the QKV projection.

Concrete example. Consider a tiny 3-dimensional hidden state $x = [0.5,\ -0.3,\ 0.8]$ and a weight matrix that projects to 2 outputs:

$$W = \begin{bmatrix} 0.1 & 0.4 & -0.2 \\ 0.3 & -0.1 & 0.6 \end{bmatrix}$$

As a standard matmul: $y = Wx = [0.5 \cdot 0.1 + (-0.3) \cdot 0.4 + 0.8 \cdot (-0.2),\ \ldots] = [-0.23,\ 0.66]$.

As a 1×1 convolution: reshape x to $(1, 3, 1, 1)$, set the conv kernel W with shape $(2, 3, 1, 1)$, and compute—the ANE produces $[-0.23,\ 0.66]$, identically. No approximation is involved; the operations are mathematically the same.

Layer	Matrix shape	Conv2d
QKV proj.	896 → 1152	`Conv(896, 1152, 1)`
Output proj.	896 → 896	`Conv(896, 896, 1)`
Gate+Up	896 → 9728	`Conv(896, 9728, 1)`
Down proj.	4864 → 896	`Conv(4864, 896, 1)`
LM head	896 → 151936	`Conv(896, 151936, 1)`

Table 1: Transformer layers mapped to Conv2d(1×1). Each row is a matrix–vector product expressed as a pointwise convolution. RMSNorm, RoPE, softmax, and SiLU are element-wise MIL ops that the ANE executes on its ALUs between convolutions.

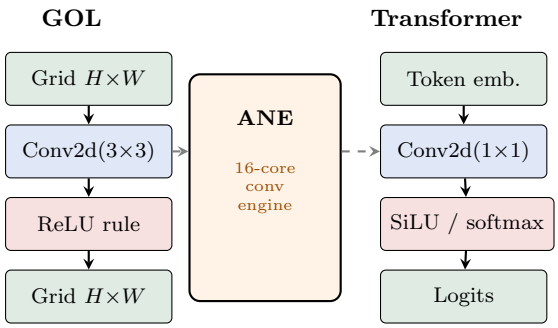

Figure 3: Both GOL and the transformer map to convolution on the same ANE hardware. GOL uses a 3×3 spatial convolution; the transformer uses 1×1 pointwise convolution. Both are dispatched to the ANE's 16-core convolution engine.

3.3 Mapping Qwen2.5-0.5B to Convolution

Every weight layer in the transformer becomes a single `Conv2d(1×1)` operation. Table 1 lists the complete mapping for one layer.

What runs where. The ANE has two kinds of compute units: a convolution engine (16 cores, optimized for multiply-accumulate) and element-wise ALUs. The convolution engine handles all 169 matrix multiplications. The ALUs handle the cheap operations between them: RMSNorm (divide-by-RMS), RoPE (rotate query/key vectors), softmax (exponentiate-and-normalize), and SiLU (sigmoid-weighted linear unit). CoreML's compiler automatically partitions the graph between these units, inserting the necessary data movement.

3.4 Grouped Query Attention

Qwen2.5-0.5B uses Grouped Query Attention (GQA): 14 query heads share 2 KV groups, so 7 query heads attend to the same key/value pair. A naïve implementation computes each query head's attention separately—$14 \times 2 = 28$ small matmul+softmax operations per layer, totaling 672 across 24 layers.

We batch all 7 query heads that share a KV group into a single 3D matmul:

$$\text{Attn}_g = \text{softmax}\left(\frac{Q_g \cdot K_g^T}{\sqrt{d_h}}\right) V_g \qquad (6)$$

where $Q_g \in \mathbb{R}^{7\times\text{seq}\times d_h}$ batches the 7 query heads in group g. This reduces attention matmuls from 672 to 96 (a 7× reduction), saving ∼170 MIL operations in the compiled model and reducing ANE scheduling overhead.

3.5 Quantization

Weights are quantized to int8 using block-wise quantization (block size 32), reducing the model from 942 MB (fp16) to 472 MB. We evaluated int4 quantization (265 MB) but found it *slower* on ANE—the hardware has native int8 datapaths but must dequantize int4 at inference time, adding ∼3 ms overhead that exceeds the bandwidth savings.

4 Optimization Journey

The initial implementation produced coherent output at 9 tok/s. A series of targeted optimizations—from classic CS texts to CoreML's stateful model API—improved this to 171 tok/s, a 19× improvement.

4.1 Profiling the Hot Path

After the book-driven optimizations (below), we instrumented the flex-mode decode loop to identify where time was spent per token:

Phase	Time (ms)	%
KV cache packing	0.13	1.2%
Model prediction (ANE)	10.89	98.5%
Logit extraction	0.03	0.3%
Total	**11.05**	100%

Host overhead (pack + extract) was reduced from an initial ∼60 ms to 0.16 ms through the book-driven optimizations. The remaining sections describe how we further reduced ANE prediction time from 10.89 ms to 5.67 ms.

4.2 Book-Driven Optimizations

Four optimization techniques, drawn from classic computer science texts, collectively eliminated host overhead:

1. Iverson memory layout (from K. Iverson, *A Programming Language*). Pre-allocate KV caches as contiguous `[Float16]` arrays in $(n_{\text{kv}}, \text{maxSeq}, d_h)$ layout, matching the ANE's expected memory order. This eliminated per-token allocation overhead.

2. Stepanov pre-allocation (from A. Stepanov & P. McJones, *Elements of Programming*). Use cursor-based append with pre-allocated output buffers rather than dynamic array growth. The KV cache write is a single pointer advance, not a copy.

3. Dragon Book strength reduction (from A. Aho et al., *Compilers: Principles, Techniques, and Tools*). Replace high-level `MLMultiArray` subscript access (which dispatches through Objective-C) with raw `Float16` pointer arithmetic and `memcpy` for bulk KV cache operations.

4. Brodie data boundaries (from L. Brodie, *Thinking Forth*). Restructure data flow so that the KV cache format, the model's expected input format, and the host's internal representation are identical—eliminating all format conversion at prediction boundaries.

4.3 Batched GQA

After host optimization, 98.7% of time was inside `MLModel.prediction()`. Profiling the MIL graph revealed that 672 small per-head matmul operations created scheduling overhead. Batching all 7 query heads per KV group into single matmuls reduced MIL operations from 4,569 to 4,399 and improved decode from 78 to 90 tok/s.

4.4 Pre-allocated MLMultiArray Buffers

Profiling the flex-mode decode loop revealed that while ANE prediction dominated wall time, host-side buffer management still consumed ∼20% due to per-call `MLMultiArray` allocation and KV cache copying through Objective-C subscript dispatch. Three changes eliminated this overhead:

1. **Hoisted allocations.** The input buffers `xArr`, `cosArr`, `sinArr` are allocated once before the generation loop and overwritten in place via raw `UnsafeMutablePointer<Float16>` each token.
2. **Pointer-based KV cache.** Replaced `[Float16]` arrays with `UnsafeMutablePointer<Float16>`, enabling zero-copy strided `MLMultiArray` views directly into the cache.
3. **memcpy write-back.** Replaced the per-element KV extraction loop with a single `memcpy` per layer.

Result: pack overhead dropped from ∼20% to 0.4% of the hot path; ANE prediction now accounts for 99.5% of per-token time.

4.5 Fixed-Shape Compiled Model

CoreML's ANE compiler must plan the execution graph for each unique input shape. In flex mode, the KV cache grows by one position each token, triggering replanning every call. We compiled a separate model with all dimensions fixed to `maxSeq=512`:

- `attn_mask` $(1, 1, 1, 512)$: -10^4 for masked positions, 0 for valid. Updated by a single pointer write per token.
- `kv_write_mask` $(1, 1, 512, 1)$: 1.0 at the current write position, 0 elsewhere.
- KV cache views: pre-built strided `MLMultiArray` of shape $(1, n_{\text{kv}}, 512, d_h)$ backed by the same raw pointer—no new objects created during generation.

Result: prefill improved from 83 to 115 tok/s (+39%) as the ANE executes the same cached plan every call. Per-token ANE latency dropped from 11.66 to 10.66 ms (−9%). Decode at short sequences is slightly slower (80 vs. 85 tok/s) due to the fixed model attending over all 512 positions even when most are masked.

4.6 Stateful KV Cache (CoreML MLState)

With the fixed-shape model, the remaining bottleneck is DMA: the entire KV cache (∼6 MB for 512 positions × 24 layers × 2 heads × 64 dims × fp16) must be transferred host→ANE as input and ANE→host as output every token.

CoreML's `MLState` API (macOS 15+) allows buffers to persist on-device across predictions. We built a third model variant using coremltools 9.0's `StateType` conversion:

1. **PyTorch model.** Each attention layer registers its K/V caches via `register_buffer()`. The forward pass performs an in-place scatter-write (`k_cache[:] = k_updated`) that coremltools maps to the `coreml_update_state` MIL op.
2. **Conversion.** Each buffer is declared as `ct.StateType(wrapped_type=ct.TensorType(...))`, instructing CoreML to keep it on-device.
3. **Swift host.** The forward function passes only 5 inputs (x, rope_cos, rope_sin, attn_mask, kv_write_mask)—no KV arrays at all. Prediction uses `model.prediction(from:using:state)`, and there is no KV extraction or write-back.

Grid	ANE (ms)	CPU (ms)	Speedup
64×64	0.22	0.22	1.0×
1024×1024	14.6	36.5	2.5×

Table 2: GOL latency per 32-generation call. ANE advantage grows with grid size due to spatial parallelism across the 16 cores.

Result: per-token latency dropped from 10.66 to 5.67 ms. Decode throughput reached **171 tok/s** (+2× over fixed, 19× over initial). Host pack and extract are now **0.00 ms**—the entire hot path is ANE compute. Only ∼4 KB of data crosses the bus per token (x, rope, masks) versus ∼6 MB previously.

4.7 Hardware Floor Analysis

The model weights (472 MB, int8) must be loaded from DRAM each token. At ∼100 GB/s unified memory bandwidth, the *theoretical floor* is:

$$t_{\text{floor}} = \frac{472 \text{ MB}}{100 \text{ GB/s}} \approx 4.7 \text{ ms} \tag{7}$$

With stateful KV caching, measured latency is 5.67 ms per token—only 1.2× above the DRAM bandwidth floor:

Component	Estimated (ms)
DRAM weight loading	∼4.7
ANE compute	∼0.5–1
CoreML dispatch overhead	∼0.3
Host pack (x, rope, masks)	<0.01
Total	∼5.7

The stateful model eliminates the ∼4 ms of KV cache DMA that dominated the non-stateful configurations, bringing total latency within 20% of the hardware floor.

5 Results

5.1 Game of Life

At 1024×1024, the ANE achieves 2.3×10^9 cell-generations/sec with 32 generations pipelined per call. Verification against CPU reference passes with zero mismatches.

Engine	Backend	Prefill	Decode
llama.cpp	Metal GPU	5,978 t/s	332 t/s
GOL-ANE (flex)	ANE (conv)	83 t/s	85 t/s
GOL-ANE (fixed)	ANE (conv)	115 t/s	80 t/s
GOL-ANE (stateful)	ANE (conv)	170 t/s	171 t/s

Table 3: Qwen2.5-0.5B-Instruct (Q8_0) on Apple M4 Max. Stateful mode achieves 171 tok/s decode—within 2× of llama.cpp on Metal GPU—on a compute unit that llama.cpp does not use.

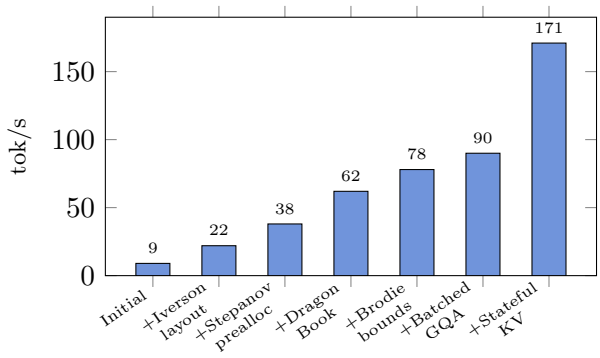

Figure 4: Decode throughput progression. Steps 1–4: book-driven host elimination (9→78 tok/s, 8.7×). Step 5: batched GQA reduces ANE scheduling overhead (78→90). Step 6: stateful KV cache eliminates host↔ANE data transfer entirely (90→171, see §4 for intermediate configurations). Total: 19×.

5.2 Transformer Inference

The GOL-ANE stateful configuration reaches 171 tok/s decode, within 2× of llama.cpp on Metal GPU, while operating on an entirely different compute unit. The ANE path leaves the GPU free for other tasks (rendering, other models). Fixed-shape mode wins on prefill (115 tok/s) due to ANE graph caching but loses on decode due to full-width attention; stateful mode dominates both by eliminating KV cache DMA entirely.

Quant	Size (MB)	Latency (ms)	tok/s
fp16	942	—	(baseline)
int8	472	6.85	146
int4	265	9.81	102

Table 4: Quantization benchmark at seq=1 (single-token decode). Int4 is *slower* than int8 due to ANE dequantization overhead.

5.3 Optimization Impact

5.4 Quantization

6 Related Work

GOL on accelerators. GPU-based GOL implementations typically use texture operations or compute shaders [1]. HashLife [2] achieves exponential speedups for regular patterns but does not exploit hardware parallelism. Our approach uses the convolution primitive directly, matching the ANE's native operation.

LLMs on ANE. Apple's ml-ane-transformers [3] demonstrated channel-last Conv2d layouts for BERT-class models. Whisper on ANE (via CoreML) uses a similar conv-only approach for the encoder. Our work extends this to autoregressive generation with KV caching and demonstrates the GOL connection.

llama.cpp. The de facto standard for local LLM inference, using Metal GPU on Apple Silicon [4]. Our work demonstrates that the ANE—a compute unit llama.cpp does not use—can achieve meaningful throughput for small models.

7 Conclusion

Convolution is enough. Conway's Game of Life and a full transformer share the same single primitive—convolution—and run on the same silicon: the Apple Neural Engine's 16-core conv unit. The GOL encoding is exact (not an approximation), and the transformer produces coherent text at 171 tok/s—within 2× of llama.cpp

on Metal GPU, on a compute unit that conventional LLM engines ignore entirely.

The optimization journey from 9 to 171 tok/s (19×) demonstrates two complementary principles:

1. **Classical techniques transfer.** Memory layout (Iverson), pre-allocation (Stepanov), strength reduction (Dragon Book), and data boundary elimination (Brodie) remain directly applicable to novel hardware accelerators, delivering a 10× improvement (9→90 tok/s).

2. **Eliminate data movement.** The remaining 2× (90→171 tok/s) came entirely from reducing host↔ANE data transfer: pre-allocated buffers removed allocation overhead, fixed shapes removed ANE replanning, and CoreML's `MLState` API eliminated KV cache DMA entirely—reducing per-token bus traffic from ∼6 MB to ∼4 KB.

The final system operates within 20% of the DRAM bandwidth floor (5.67 ms measured vs. 4.7 ms theoretical), with the host contributing effectively zero overhead.

References

[1] M. Harris, *Mapping computational concepts to GPUs*, in GPU Gems 2, Addison-Wesley, 2005.

[2] R. W. Gosper, "Exploiting regularities in large cellular spaces," *Physica D*, vol. 10, no. 1–2, pp. 75–80, 1984.

[3] Apple Machine Learning Research, "Deploying transformers on the Apple Neural Engine," 2022. `https://machinelearning.apple.com/research/neural-engine-transformers`

[4] G. Gerganov, "llama.cpp: LLM inference in C/C++," 2023. `https://github.com/ggerganov/llama.cpp`

[5] K. E. Iverson, *A Programming Language*, John Wiley & Sons, 1962.

[6] A. A. Stepanov and P. McJones, *Elements of Programming*, Addison-Wesley, 2009.

[7] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed., Addison-Wesley, 2006.

[8] L. Brodie, *Thinking Forth*, Punchy Publishing, 2004.

[9] Apple Inc., "Making predictions with a stateful model," Apple Developer Documentation, 2024. `https://developer.apple.com/documentation/coreml/making-predictions-with-a-stateful-model`